Module 11.4 - Lazy Loading

> **Learning Objectives:** Master lazy loading modules for performance optimization, implement preloading strategies, understand bundle optimization techniques, and create efficient module loading patterns in Angular applications.

---

1. Understanding Lazy Loading

> **Lazy Loading Approaches:** This module covers lazy loading in both module-based and standalone component applications. Standalone components make lazy loading even simpler with less boilerplate code.

1.1 What is Lazy Loading?

Lazy loading is a design pattern that defers loading of modules until they are needed. Instead of loading the entire application at startup, modules are loaded on-demand when users navigate to specific routes.

Benefits of Lazy Loading:

- **Faster Initial Load:** Smaller initial bundle size
- **Better Performance:** Load only what users need
- **Reduced Bandwidth:** Users don't download unused features
- **Improved UX:** Application becomes interactive faster
- **Scalability:** Large apps remain performant

1.2 Eager Loading vs Lazy Loading

Aspect	Eager Loading	Lazy Loading
Loading Time	At application startup	On-demand (when route accessed)
Initial Bundle	Large (all modules)	Small (core modules only)

Aspect	Eager Loading	Lazy Loading
First Paint	Slower	Faster
Navigation	Instant (already loaded)	Slight delay first time
Bundle Files	Single main.js	Multiple chunk files
Use For	Core features	Feature modules

1.3 When to Use Lazy Loading

Good Candidates for Lazy Loading:

- Admin panels (only for admin users)
- User dashboards (only after login)
- Reports and analytics modules
- Settings and configuration pages
- Feature modules with heavy dependencies
- Rarely accessed functionality

NOT Good for Lazy Loading:

- Landing pages (need immediate display)
- Core navigation components
- Shared services and utilities
- Frequently accessed features
- Small modules (overhead not worth it)

1.4 Module-Based vs Standalone Lazy Loading

Aspect	Module-Based	Standalone
Setup	Require feature module + routing module	Just component files and routes
Syntax	`loadChildren: () => import(...).then(m => m.Module)`	`loadComponent` or `loadChildren` with routes
Boilerplate	More files and configuration	Minimal configuration
Bundle Size	Includes module wrapper	Smaller chunks (no module overhead)
Flexibility	Load entire module	Load individual components or route trees
Recommended For	Existing applications	New applications (Angular 14+)

---

2. Implementing Lazy Loading

2.1 Module-Based Lazy Loading (Traditional)

Step 1: Generate Feature Module with CLI

```
ng generate module features/admin --route admin --module app-routing.module
```

This command creates:

- AdminModule with routing
- Lazy loading route configuration in app-routing.module
- Admin component as default route

How It Works: A feature module groups related components and defines its own routes. When lazy loaded, Angular downloads this module as a separate JavaScript file only when the user navigates to its route.

Admin Module (admin.module.ts):

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterModule, Routes } from '@angular/router';
import { AdminDashboardComponent } from './admin-dashboard.component';
import { UserManagementComponent } from './user-management.component';

const routes: Routes = [
  { path: '', component: AdminDashboardComponent },
  { path: 'users', component: UserManagementComponent }
];

@NgModule({
  declarations: [AdminDashboardComponent, UserManagementComponent],
  imports: [CommonModule, RouterModule.forChild(routes)]
})
export class AdminModule {
  constructor() {
    console.log('AdminModule loaded!');
  }
}
```

Key Points:

- `CommonModule` - Use this instead of BrowserModule in feature modules
- `RouterModule.forChild(routes)` - Registers child routes for this module
- `constructor` - Logs when module loads (helps debug lazy loading)
- Empty path ('') - Becomes the default route within the admin section

Step 2: Configure Lazy Loading in App Routing:

How loadChildren Works: The `loadChildren` property uses dynamic import syntax `() => import('./path')`. This tells Webpack to create a separate bundle for the AdminModule. The `.then(m => m.AdminModule)` extracts the module from the imported file once it loads.

App Routing (app-routing.module.ts):

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';

const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'admin',
    loadChildren: () => import('./features/admin/admin.module')
      .then(m => m.AdminModule)
  }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

What Happens:

1. App starts: Only HomeComponent loads in main bundle
2. User navigates to /admin: Angular downloads admin-module.js
3. Module loads: Constructor runs, routes register
4. Navigation completes: AdminDashboardComponent displays

> **Application Startup:**
>
> 1. App loads: main.js (200 KB) - Core Angular + AppModule
> 2. User sees home page immediately
>
> **User navigates to /admin:**
>
> 1. Router detects lazy route
> 2. Downloads: admin-module.js (150 KB)
> 3. Console: AdminModule loaded!
> 4. Admin dashboard displays
>
> **User navigates to /admin/users:**

```
1. Module already loaded (no download)
2. UserManagementComponent displays immediately

Result:
✓ Initial load: 200 KB (fast)
✓ Admin module: Only loaded when needed
✓ Subsequent admin navigation: Instant
```

2.2 Lazy Loading with Guards

Why Use Guards with Lazy Loading: Guards prevent unauthorized users from even downloading the module code. This improves security (users can't inspect admin code) and saves bandwidth.

Route Configuration:

```
const routes: Routes = [
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule),
    canLoad: [AdminGuard],      // Checks BEFORE downloading
    canActivate: [AuthGuard]    // Checks AFTER downloading
  }
];
```

Guard Differences:

- `canLoad` - Runs before module download. If false, module never downloads
- `canActivate` - Runs after module loads. Protects individual routes within module

Example Guard:

```
@Injectable({ providedIn: 'root' })
export class AdminGuard implements CanLoad {
  constructor(private authService: AuthService, private router: Router) {}

  canLoad(): boolean {
    if (this.authService.isAdmin()) {
      return true;  // Allow download
    }
```

```
    this.router.navigate(['/login']);
    return false;  // Block download
  }
}
```

2.3 Multiple Lazy-Loaded Modules

Building on Previous Example: Now we add more lazy modules. Each module downloads independently when its route is accessed.

Multiple Lazy Modules:

```
const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },

  // Lazy loaded modules
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule)
  },
  {
    path: 'user',
    loadChildren: () => import('./user/user.module').then(m => m.UserModule)
  },
  {
    path: 'products',
    loadChildren: () => import('./products/products.module').then(m => m.ProductsModule)
  },

  { path: '**', redirectTo: '' }
];
```

How Multiple Modules Work:

- Each `loadChildren` creates a separate JavaScript bundle
- Bundles download independently based on user navigation
- Once downloaded, a module stays in memory for instant re-navigation

2.4 Shared Modules in Lazy Loading

Why Shared Modules: When multiple lazy modules use the same components (like buttons, forms, pipes), create a SharedModule to avoid code duplication. Each lazy module imports it as needed.

Shared Module (shared.module.ts):

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { LoadingSpinnerComponent } from './loading-spinner.component';
import { HighlightDirective } from './highlight.directive';

@NgModule({
  declarations: [LoadingSpinnerComponent, HighlightDirective],
  imports: [CommonModule],
  exports: [LoadingSpinnerComponent, HighlightDirective, CommonModule]
})
export class SharedModule {}
```

Using in Lazy Module:

```
import { NgModule } from '@angular/core';
import { SharedModule } from '../../shared/shared.module';
import { RouterModule, Routes } from '@angular/router';
import { AdminDashboardComponent } from './admin-dashboard.component';

const routes: Routes = [
  { path: '', component: AdminDashboardComponent }
];

@NgModule({
  declarations: [AdminDashboardComponent],
  imports: [SharedModule, RouterModule.forChild(routes)]
})
export class AdminModule {}
```

What Happens: The SharedModule code is included in EACH lazy module that imports it. This is intentional - Angular bundles it with each module so they remain independent.

declarations: [// components], imports: [SharedModule, // Use shared module instead of importing everything separately RouterModule.forChild(routes)] }) export class AdminModule {}

3. Preloading Strategies

3.1 Understanding Preloading

Preloading loads lazy modules in the background after the initial application load, providing the best of both worlds: fast initial load and instant navigation.

Loading Strategies Comparison:

Strategy	Initial Load	Navigation	Use Case
Eager	Slow	Instant	Core features
Lazy	Fast	Delay first time	Rarely used features
Preload All	Fast	Instant (after preload)	Small apps
Custom Preload	Fast	Instant (selected modules)	Selective optimization

3.2 PreloadAllModules Strategy

The simplest preloading strategy works with both module-based and standalone routes.

Module-Based App Routing with PreloadAllModules:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes, PreloadAllModules } from '@angular/router';

const routes: Routes = [
  { path: '', component: HomeComponent },

  {
    path: 'admin',
    loadChildren: () => import('./features/admin/admin.module')
      .then(m => m.AdminModule)
  },
  {
    path: 'user',
    loadChildren: () => import('./features/user/user.module')
      .then(m => m.UserModule)
```

```
  }
];

@NgModule({
  imports: [RouterModule.forRoot(routes, {
    preloadingStrategy: PreloadAllModules  // Preload all lazy modules
  })],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

Loading Timeline with PreloadAllModules:

t=0ms: User opens application
→ Download main.js (200 KB)
→ Display home page

t=500ms: App initialized, user can interact
→ Start preloading in background
→ Download admin-module.js (150 KB)
→ Download user-module.js (100 KB)

t=1500ms: Preloading complete
→ All modules ready

User clicks "Admin":
→ Instant navigation (already loaded)
→ No download delay

Benefits:
✓ Fast initial load
✓ Instant navigation after preload
✓ Better user experience

3.3 NoPreloading Strategy

What It Does: Default behavior - no preloading, modules load only when user navigates to them.

```
import { NoPreloading } from '@angular/router';

@NgModule({
  imports: [RouterModule.forRoot(routes, {
    preloadingStrategy: NoPreloading  // Explicit no preloading
  })],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

Use When: You have many large modules and want maximum control over loading.

3.4 Custom Preloading Strategy

Why Custom Strategies: PreloadAllModules might download too much. Custom strategies let you choose WHICH modules to preload based on criteria like user role, priority, or timing.

Custom Strategy (selective-preload.strategy.ts):

```
import { Injectable } from '@angular/core';
import { PreloadingStrategy, Route } from '@angular/router';
import { Observable, of, timer } from 'rxjs';
import { mergeMap } from 'rxjs/operators';

@Injectable({ providedIn: 'root' })
export class SelectivePreloadStrategy implements PreloadingStrategy {
  preload(route: Route, load: () => Observable<any>): Observable<any> {
    // Check route data for preload flag
    if (route.data && route.data['preload']) {
      const delay = route.data['delay'] || 0;
      console.log(`Preloading: ${route.path}`);
      return timer(delay).pipe(mergeMap(() => load()));
    }
    console.log(`Not preloading: ${route.path}`);
    return of(null);  // Don't preload
  }
}
```

Marking Routes for Preload:

```
const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule)
    // No preload - loads on demand
  },
  {
    path: 'user',
    loadChildren: () => import('./user/user.module').then(m => m.UserModule),
    data: { preload: true }  // Preload immediately
  },
  {
    path: 'products',
    loadChildren: () => import('./products/products.module').then(m => m.ProductsModule),
    data: { preload: true, delay: 2000 }  // Preload after 2 seconds
  }
];

@NgModule({
  imports: [RouterModule.forRoot(routes, {
    preloadingStrategy: SelectivePreloadStrategy
  })],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

How It Works:

- Strategy checks each route's `data.preload` property
- If true, calls `load()` to download the module
- If false or missing, returns `of(null)` (no download)
- Optional delay allows staggered loading

3.5 Network-Aware Preloading Strategy

Preload based on user's network conditions.

Network-Aware Preload Strategy:

```
import { Injectable } from '@angular/core';
import { PreloadingStrategy, Route } from '@angular/router';
import { Observable, of } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class NetworkAwarePreloadStrategy implements PreloadingStrategy {
  preload(route: Route, load: () => Observable<any>): Observable<any> {
    // Check if preload flag is set
    if (!route.data || !route.data['preload']) {
      return of(null);
    }

    // Check network connection
    const connection = (navigator as any).connection;

    if (!connection) {
      // Network API not supported, preload anyway
      console.log('Preloading (network info unavailable):', route.path);
      return load();
    }

    // Get effective connection type
    const effectiveType = connection.effectiveType;

    console.log('Network type:', effectiveType);

    // Preload only on fast connections
    if (effectiveType === '4g' || effectiveType === '3g') {
      console.log('✓ Preloading on fast connection:', route.path);
      return load();
    }

    // Don't preload on slow connections
    console.log('✗ Skipping preload on slow connection:', route.path);
    return of(null);
  }
}
```

Usage:

```
@NgModule({
  imports: [RouterModule.forRoot(routes, {
    preloadingStrategy: NetworkAwarePreloadStrategy
  })],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

3.6 Predictive Preloading

The Idea: If a user hovers over a link, they're likely to click it. Start downloading the module during the hover to make navigation feel instant.

Hover-to-Preload Directive:

```
import { Directive, HostListener, Input } from '@angular/core';
import { Router } from '@angular/router';

@Directive({
  selector: '[appPreloadOnHover]',
  standalone: true
})
export class PreloadOnHoverDirective {
  @Input() appPreloadOnHover: string = '';  // Route to preload

  constructor(private router: Router) {}

  @HostListener('mouseenter')
  onHover() {
    console.log('Preloading on hover:', this.appPreloadOnHover);
    this.router.navigate([this.appPreloadOnHover], { skipLocationChange: true });
  }
}
```

Usage:

```
<nav>
  <a routerLink="/admin" appPreloadOnHover="/admin">Admin</a>
  <a routerLink="/products" appPreloadOnHover="/products">Products</a>
</nav>
```

User Experience:

1. User hovers over "Admin" link → Module starts downloading
2. User clicks link (typically 200-500ms later) → Module ready, instant navigation
3. User moves mouse away without clicking → Wasted bandwidth, but small cost

---

4. Bundle Analysis and Optimization

4.1 Analyzing Bundle Size

Why Analyze: Know which modules are too large and need optimization. Angular CLI shows bundle sizes after each build.

Basic Analysis:

```
ng build
```

Output shows:

```
main.js          250 KB  (initial bundle - core app)
admin-module.js  150 KB  (lazy - admin features)
user-module.js   100 KB  (lazy - user features)
```

Set Budget Warnings (angular.json):

```
{
  "projects": {
    "your-app": {
      "architect": {
```

```
        "build": {
          "configurations": {
            "production": {
              "budgets": [
                {
                  "type": "initial",
                  "maximumWarning": "500kb",
                  "maximumError": "1mb"
                },
                {
                  "type": "anyComponentStyle",
                  "maximumWarning": "4kb"
                }
              ]
            }
          }
        }
      }
    }
  }
}
```

What Budgets Do: Angular warns or fails the build if bundles exceed limits. Prevents accidentally shipping huge files.

}}

```
Build Output with Budgets:

Initial chunk: 450 KB (within budget)
admin-module: 150 KB
user-module: 100 KB
products-module: 200 KB

Warning: products-module exceeds recommended size
Consider splitting into smaller modules

Total Size:
Initial load: 450 KB ✓
Lazy modules: 450 KB (loaded on demand)
```

4.2 Code Splitting Best Practices

The Goal: Keep initial bundle small, split features into logical chunks.

Simple Structure:

```
src/app/
├── core/              (Eager - load at startup)
│   ├── auth.service.ts
│   └── http.interceptor.ts
│
├── shared/            (Reusable - imported as needed)
│   ├── button.component.ts
│   └── shared.module.ts
│
├── admin/             (Lazy - loads on /admin)
│   ├── admin.component.ts
│   └── admin.module.ts
│
├── user/              (Lazy - loads on /user)
│   ├── user.component.ts
│   └── user.module.ts
│
└── app-routing.module.ts
```

Key Principles:

- **Core:** Singleton services needed everywhere (auth, logging)
- **Shared:** UI components used by multiple features
- **Features:** Complete features that can load independently
- **Size target:** Keep each lazy module under 200 KB

4.3 Avoiding Common Pitfalls

> **Pitfall #1: BrowserModule in Lazy Modules**
>
> **Problem:** BrowserModule should only be imported once in AppModule. Lazy modules should use CommonModule.

```
// ❌ WRONG - Don't use BrowserModule
import { BrowserModule } from '@angular/platform-browser';
@NgModule({
  imports: [BrowserModule]
})
export class AdminModule {}

// ☑ CORRECT - Use CommonModule
import { CommonModule } from '@angular/common';
@NgModule({
  imports: [CommonModule]
})
export class AdminModule {}
```

Pitfall #2: Providing Services in Lazy Modules

Problem: Providing a service in a lazy module creates a NEW instance of that service, breaking singleton pattern.

```
// ❌ WRONG - Creates multiple instances
@NgModule({
  providers: [UserService]  // New instance per lazy module!
})
export class AdminModule {}

// ☑ CORRECT - Use providedIn: 'root'
@Injectable({ providedIn: 'root' })
export class UserService {}
```

Why: With `providedIn: 'root'`, Angular creates ONE instance shared across all modules.

Pitfall #3: Large Libraries in AppModule

Problem: Importing heavy libraries in AppModule forces them into the initial bundle.

```
// ❌ WRONG - Chart library in main bundle
import { NgChartsModule } from 'ng2-charts';
@NgModule({
  imports: [NgChartsModule]  // 300 KB added to startup!
})
export class AppModule {}

// ☑ CORRECT - Import only in feature that needs it
@NgModule({
  imports: [NgChartsModule]  // Only downloads with reports module
})
export class ReportsModule {}
```

---

5. Real-World Example: E-Commerce Application

5.1 Complete Lazy Loading Implementation

Putting It All Together: This example shows how to structure a real e-commerce app with strategic lazy loading.

Strategy:

- **Eager:** Home, catalog (public pages everyone sees)
- **Lazy + Preload:** Cart, user dashboard (logged-in users likely need)
- **Lazy + On-demand:** Admin, reports (few users, heavy modules)

App Routing:

```
import { RouterModule, Routes } from '@angular/router';
import { SelectivePreloadStrategy } from './selective-preload.strategy';
import { HomeComponent } from './home.component';

const routes: Routes = [
  // Eager - loads immediately
  { path: '', component: HomeComponent },

  // Lazy + Preload - downloads after app loads
```

```
  {
    path: 'cart',
    loadChildren: () => import('./cart/cart.module').then(m => m.CartModule),
    data: { preload: true }
  },
  {
    path: 'user',
    loadChildren: () => import('./user/user.module').then(m => m.UserModule),
    data: { preload: true }
  },

  // Lazy only - downloads when accessed
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule),
    canLoad: [AdminGuard]
  }
];

export const AppRoutingModule = RouterModule.forRoot(routes, {
  preloadingStrategy: SelectivePreloadStrategy
});
```

User Module Example:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { SharedModule } from '../shared/shared.module';
import { UserDashboardComponent } from './user-dashboard.component';
import { OrdersComponent } from './orders.component';

const routes: Routes = [
  { path: '', component: UserDashboardComponent },
  { path: 'orders', component: OrdersComponent }
];

@NgModule({
  declarations: [UserDashboardComponent, OrdersComponent],
  imports: [SharedModule, RouterModule.forChild(routes)]
})
export class UserModule {}
```

5.2 Loading Timeline

```
User Journey - Guest Visitor:

t=0ms: User visits website
→ Download: main.js (250 KB)
→ Display: Home page

t=500ms: App ready
→ Start preloading: cart-module.js
→ cart-module.js (80 KB) downloaded

User browses catalog: Instant (eager loaded)

User clicks "Add to Cart":
→ Navigate to /cart
→ Instant (already preloaded)

User Journey - Logged In User:

t=0ms: User logs in
→ Already has main.js cached

t=300ms: Dashboard displayed
→ Start preloading: user-module.js, cart-module.js
→ Both modules downloaded

User clicks "My Orders":
→ Navigate to /user/orders
→ Instant (already preloaded)

Admin Journey:

Admin navigates to /admin:
→ AdminLoadGuard checks permission
→ Download: admin-module.js (200 KB)
→ AdminModule loaded
→ Dashboard displayed

Regular users never download admin module!
```

6. Performance Monitoring

Why Monitor: Measure how long modules take to download and how lazy loading impacts user experience.

6.1 Simple Performance Tracking

Track Navigation Time:

```
import { Injectable } from '@angular/core';
import { Router, NavigationStart, NavigationEnd } from '@angular/router';

@Injectable({ providedIn: 'root' })
export class PerformanceMonitorService {
  private navStart: number = 0;

  constructor(private router: Router) {
    this.router.events.subscribe(event => {
      if (event instanceof NavigationStart) {
        this.navStart = Date.now();
      }
      if (event instanceof NavigationEnd) {
        const duration = Date.now() - this.navStart;
        console.log(`Navigation to ${event.url} took ${duration}ms`);
      }
    });
  }
}
```

Output Example:

```
Navigation to /admin took 450ms  // First time (download + render)
Navigation to /admin/users took 12ms  // Already loaded (instant)
```

What to Look For:

- First navigation to lazy route: Should be under 1 second
- Subsequent navigation: Should be under 100ms
- If slow: Module is too large, consider splitting further

7. Best Practices Summary

7.1 Lazy Loading Guidelines

Do:

- **Use standalone components** for new projects - simpler lazy loading
- Lazy load feature modules/routes, not core functionality
- Use meaningful names for generated chunks
- Keep module/route chunk size reasonable (100-200 KB target)
- Use preloading for frequently accessed features
- Combine lazy loading with guards (CanMatch for standalone)
- Monitor bundle sizes with budgets
- Test loading performance on slow connections

Don't:

- Don't over-split into too many small chunks (HTTP overhead)
- Don't lazy load landing pages or critical features
- Don't import BrowserModule in lazy modules (CommonModule instead)
- Don't provide singleton services in lazy-loaded features
- Don't preload everything (defeats the purpose)
- Don't forget to test on real network conditions

7.2 Module-Based vs Standalone Lazy Loading

Aspect	Module-Based	Standalone	Recommendation
Setup Complexity	High (module + routing module)	Low (just components + routes)	\u2713 Standalone

Aspect	Module-Based	Standalone	Recommendation
Bundle Size	Larger (includes module metadata)	Smaller (no module overhead)	\u2713 Standalone
Loading Granularity	Module level only	Component or route tree level	\u2713 Standalone
Code Sharing	SharedModule approach	Direct imports where needed	\u2713 Standalone
Migration Path	N/A	Can gradually convert	\u2713 Standalone
Maintenance	More files to manage	Fewer files, clearer structure	\u2713 Standalone

7.3 Preloading Strategy Selection

App Type	Strategy	Reason
Small App	PreloadAllModules	Fast initial load, instant navigation
Large App	Selective	Preload important, skip rarely used
Mobile-First	Network-Aware	Respect user's data plan
Internal Tool	PreloadAllModules	Fast connection, frequent use
Public Site	Predictive	Load on user intent

7.4 Performance Optimization Checklist

1. ✓ Use standalone components for new projects
2. ✓ Lazy load feature routes/modules
3. ✓ Set appropriate preloading strategy
4. ✓ Configure bundle budgets
5. ✓ Analyze bundle size regularly
6. ✓ Use shared utilities efficiently

7. ✓ Avoid importing heavy libraries in core
8. ✓ Test on slow 3G connection
9. ✓ Monitor loading metrics
10. ✓ Keep chunks focused and small
11. ✓ Use CanMatch for route protection
12. ✓ Consider code splitting within routes

---

8. Standalone Components Lazy Loading (Angular 14+)

What Changed: Standalone components eliminate the need for NgModules, making lazy loading simpler with less boilerplate. This is the recommended approach for new Angular applications.

8.1 Creating Standalone Components

What Makes a Component Standalone: Set `standalone: true` and directly import what you need instead of declaring in a module.

Standalone Component Example:

```
// admin.component.ts
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterModule } from '@angular/router';

@Component({
  selector: 'app-admin',
  standalone: true,  // This makes it standalone!
  imports: [CommonModule, RouterModule],  // Direct imports
  template: `
    <h2>Admin Dashboard</h2>
    <nav>
      <a routerLink="users">Users</a>
      <a routerLink="settings">Settings</a>
    </nav>
    <router-outlet></router-outlet>
```

```
  `
})
export class AdminComponent {}
```

Key Differences from Module-Based:

- `standalone: true` - Makes component self-contained
- `imports` array - Import dependencies directly (no module needed)
- No `declarations` - Component declares itself

8.2 Lazy Loading Single Component

Use Case: Load a single component on demand, perfect for simple routes like profile, settings, or about pages.

App Routes (app.routes.ts):

```
import { Routes } from '@angular/router';
import { HomeComponent } from './home.component';

export const routes: Routes = [
  { path: '', component: HomeComponent },

  // Lazy load single component
  {
    path: 'profile',
    loadComponent: () => import('./profile/profile.component')
      .then(m => m.ProfileComponent)
  }
];
```

How It Works:

1. User navigates to /profile
2. Angular downloads profile.component.ts as separate bundle
3. Component loads and displays
4. No module wrapper needed!

8.3 Lazy Loading Route Tree

Use Case: Load multiple related routes together (like an entire admin section with sub-routes).

Admin Routes File (admin/admin.routes.ts):

```
import { Routes } from '@angular/router';
import { AdminComponent } from './admin.component';
import { UsersComponent } from './users.component';
import { SettingsComponent } from './settings.component';

export const ADMIN_ROUTES: Routes = [
  {
    path: '',
    component: AdminComponent,
    children: [
      { path: 'users', component: UsersComponent },
      { path: 'settings', component: SettingsComponent },
      { path: '', redirectTo: 'users', pathMatch: 'full' }
    ]
  }
];
```

Main App Routes (app.routes.ts):

```
import { Routes } from '@angular/router';
import { HomeComponent } from './home.component';

export const routes: Routes = [
  { path: '', component: HomeComponent },

  // Lazy load entire route tree
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.routes')
      .then(m => m.ADMIN_ROUTES)
  }
];
```

What Happens:

- User visits /admin → Downloads admin bundle (AdminComponent + child routes)

- User visits /admin/users → Already loaded, instant navigation
- One bundle contains: AdminComponent, UsersComponent, SettingsComponent

8.4 Bootstrapping Standalone App

No AppModule Needed: Bootstrap directly with the root component and configure routing with providers.

Main Entry Point (main.ts):

```
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter, withPreloading, PreloadAllModules } from '@angular/router';
import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';

bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(
      routes,
      withPreloading(PreloadAllModules)  // Enable preloading
    )
  ]
});
```

Root App Component (app.component.ts):

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterModule],
  template: `
    <nav>
      <a routerLink="/">Home</a>
      <a routerLink="/admin">Admin</a>
      <a routerLink="/profile">Profile</a>
    </nav>
    <router-outlet></router-outlet>
```

```
  `
})
export class AppComponent {}
```

8.5 Standalone with Guards

Functional Guards: Standalone apps use functional guards (simpler than class-based guards).

Auth Guard (guards/auth.guard.ts):

```
import { inject } from '@angular/core';
import { Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const authGuard = () => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isLoggedIn()) {
    return true;
  }
  return router.parseUrl('/login');
};
```

Using Guard in Routes:

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'admin',
    canMatch: [authGuard],  // Blocks download if not authorized
    loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES)
  }
];
```

canMatch vs canActivate:

- `canMatch` - Prevents lazy module download (saves bandwidth)
- `canActivate` - Checks after module loads

- Use `canMatch` for lazy routes to prevent unauthorized downloads

8.6 Complete Standalone Example

Project Structure:

```
src/
├── app/
│   ├── home/
│   │   └── home.component.ts           (standalone, eager loaded)
│   │
│   ├── admin/
│   │   ├── admin.component.ts          (standalone - parent component)
│   │   ├── users.component.ts          (standalone - child component)
│   │   └── admin.routes.ts             (route definitions for admin)
│   │
│   ├── profile/
│   │   └── profile.component.ts        (standalone - single lazy component)
│   │
│   ├── guards/
│   │   └── auth.guard.ts               (functional guard)
│   │
│   ├── app.component.ts                (root standalone component)
│   ├── app.routes.ts                   (main application routes)
│   └── main.ts                         (bootstrap file)
│
├── index.html
└── styles.css
```

1. Auth Guard (guards/auth.guard.ts):

```
import { inject } from '@angular/core';
import { Router } from '@angular/router';

// Simple auth service (you'd have a real one)
export class AuthService {
  private loggedIn = false;

  isLoggedIn(): boolean {
    return this.loggedIn;
  }

  login() {
```

```
    this.loggedIn = true;
  }

  logout() {
    this.loggedIn = false;
  }
}

// Functional guard
export const authGuard = () => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isLoggedIn()) {
    return true;  // Allow navigation
  }

  // Redirect to login
  return router.parseUrl('/login');
};
```

2. Admin Routes (admin/admin.routes.ts):

```
import { Routes } from '@angular/router';
import { AdminComponent } from './admin.component';
import { UsersComponent } from './users.component';

export const ADMIN_ROUTES: Routes = [
  {
    path: '',
    component: AdminComponent,
    children: [
      {
        path: 'users',
        component: UsersComponent
      },
      {
        path: '',
        redirectTo: 'users',
        pathMatch: 'full'
      }
```

```
    ]
  }
];
```

3. Main App Routes (app.routes.ts):

```
// app.routes.ts
import { Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { authGuard } from './guards/auth.guard';

export const routes: Routes = [
  // Eager - loads immediately
  { path: '', component: HomeComponent },

  // Lazy load single component
  {
    path: 'profile',
    canActivate: [authGuard],
    loadComponent: () => import('./profile/profile.component')
      .then(m => m.ProfileComponent)
  },

  // Lazy load route tree with preload
  {
    path: 'admin',
    canMatch: [authGuard],
    loadChildren: () => import('./admin/admin.routes')
      .then(m => m.ADMIN_ROUTES),
    data: { preload: true }
  }
];
```

4. Home Component (home/home.component.ts):

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  standalone: true,
  template: `
    <h1>Welcome Home</h1>
    <p>This component loads immediately (eager loading)</p>
```

```
  `
})
export class HomeComponent {}
```

5. Admin Component (admin/admin.component.ts):

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';

@Component({
  selector: 'app-admin',
  standalone: true,
  imports: [RouterModule],
  template: `
    <h2>Admin Dashboard</h2>
    <nav>
      <a routerLink="users">Manage Users</a>
    </nav>
    <router-outlet></router-outlet>
  `
})
export class AdminComponent {}
```

6. Users Component (admin/users.component.ts):

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-users',
  standalone: true,
  template: `
    <h3>User Management</h3>
    <p>Manage users here...</p>
  `
})
export class UsersComponent {}
```

7. Profile Component (profile/profile.component.ts):

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-profile',
  standalone: true,
  template: `
    <h2>User Profile</h2>
    <p>Your profile information...</p>
  `
})
export class ProfileComponent {}
```

8. Root App Component (app.component.ts):

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterModule],
  template: `
    <nav>
      <a routerLink="/">Home</a>
      <a routerLink="/profile">Profile</a>
      <a routerLink="/admin">Admin</a>
    </nav>
    <router-outlet></router-outlet>
  `
})
export class AppComponent {}
```

9. Bootstrap (main.ts):

```
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter, withPreloading, PreloadAllModules } from '@angular/router';
import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';

bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(
```

```
      routes,
      withPreloading(PreloadAllModules)
    )
  ]
}).catch(err => console.error(err));
```

How It All Works Together:

1. **App starts:** main.ts bootstraps AppComponent
2. **Home loads:** HomeComponent loads immediately (eager)
3. **Navigate to /profile:** Downloads profile.component.ts bundle, displays ProfileComponent
4. **Navigate to /admin:** Downloads admin bundle (admin.component.ts + users.component.ts + admin.routes.ts)
5. **Navigate to /admin/users:** Already loaded, instant navigation

8.7 Why Standalone is Better

Aspect	Module-Based	Standalone
Files Required	3 (module, routing, component)	1-2 (component, optional routes)
Boilerplate	@NgModule decorator, imports, declarations	Just standalone: true
Bundle Size	Includes NgModule metadata	Smaller, no module overhead
Lazy Loading	loadChildren only	loadChildren OR loadComponent
Learning Curve	Must understand modules	Simpler, more intuitive

Migration Tip: You can mix module-based and standalone in the same app, allowing gradual migration.

---

9. Practice Exercises

Exercise 1: Basic Lazy Loading

- Update routing configuration to use loadComponent/loadChildren

```
src/app/
├── home/
│   └── home.component.ts
├── admin/
│   ├── admin.component.ts
│   ├── admin-dashboard.component.ts
│   ├── user-management.component.ts
│   └── admin.routes.ts
├── shop/
│   ├── shop.component.ts
│   ├── product-list.component.ts
│   ├── product-detail.component.ts
│   └── shop.routes.ts
├── app.component.ts
├── app.routes.ts
└── main.ts
```

9. Practice Exercises

Exercise 1: Basic Lazy Loading

Create an app with three modules:

- Home (eager loaded)
- Dashboard (lazy loaded)
- Admin (lazy loaded with guard)
- Compare bundle sizes before and after lazy loading

Exercise 2: Selective Preloading

Implement a custom preloading strategy:

- Preload "important" modules immediately
- Preload "optional" modules after 3 seconds
- Don't preload "admin" modules
- Test and observe console logs

Exercise 3: Real-World Application

Build a simple e-commerce app with:

- Product catalog (eager)
- Shopping cart (lazy + preload)
- Checkout (lazy, no preload)
- Admin panel (lazy, guard-protected)

Given a large application:

- Analyze current bundle sizes
- Identify optimization opportunities
- Implement lazy loading for heavy modules
- Set up bundle budgets
- Achieve 50% reduction in initial bundle

---

10. Summary

In this module, you learned:

- **Lazy Loading Basics:** Loading modules on-demand for better performance
- **Implementation:** Using loadChildren for lazy-loaded routes
- **Preloading Strategies:** PreloadAllModules, custom, network-aware
- **Bundle Optimization:** Analyzing and optimizing bundle sizes

- **Best Practices:** Module organization and common pitfalls
- **Performance Monitoring:** Measuring and tracking loading performance
- **Real-World Patterns:** E-commerce example with complete lazy loading

Lazy loading is one of the most effective optimization techniques in Angular. When combined with appropriate preloading strategies, it provides the best balance between fast initial load and smooth navigation throughout the application.

> **Next Module Preview:** In Module 12.1, we'll explore HTTP Client setup, establishing API communication with REST services, and handling HTTP requests and responses in Angular applications.

Module 11.4 - Lazy Loading | Angular Framework Training